EGER 1774
ESZTERHÁZY KÁROLY KATOLIKUS EGYETEM
Matematikai és Informatikai Intézet

Python alapú grafikus alkalmazásfejlesztés

Készítette
Mészáros Réka
Programtervező informatikus BSc

Témavezető
Kovács Ádám
Tanársegéd

Eger, 2026

Tartalomjegyzék

1. Bevezetés **5**

2. Technológiai áttekintés **7**

2.1. Programozási nyelvek 7

2.1.1. Python 7

2.1.2. SQL 8

2.2. GitHub 8

2.3. Fejlesztői környezetek 9

2.3.1. PyCharm 9

2.3.2. Visual Studio Code 9

2.3.3. PhPMyAdmin 9

2.3.4. Elérhetőségi linkek 9

2.4. Használt csomagok 10

2.4.1. Pygame könyvtár 10

2.4.2. MySQL-connector-python 10

3. Rendszerterv **12**

3.1. A rendszer célja 12

3.2. Követelmények listája 12

3.2.1. Funkcionális követelmények 12

3.2.2. Nem funkcionális követelmények 13

3.3. Használati esetek (Use case-ek) 13

3.4. Adatbázis-terv 13

3.5. UML-diagramok, folyamatábrák 13

3.5.1. Osztálydiagram 13

3.5.2. Folyamatábra 14

4. Saját szoftver megvalósítása **15**

4.1. Az alkalmazás felépítése 15

4.1.1. Architekturális áttekintés 15

4.1.2. Komponensek és azok szerepe 15

4.1.3. Adatfolyam és működési logika 19

4.2. Adatbázis 22
4.3. Frontend 23

5. Futtatás **25**

6. Tesztelés **26**
6.1. Manuális tesztelés 26
6.2. Automatikus tesztelés 26

7. Összegzés **28**

Irodalomjegyzék **30**

Ábrák jegyzéke

4.1. Hotbar materials 17
4.2. Hotbar tools 17
4.3. Kuka lezárva 18
4.4. Kuka nyitva aktív item-mel a közelében 18
4.5. Adatbázis felépítése 23
4.6. Pipetta sprite 24
4.7. Pipetta(H2O) sprite 24

1. fejezet

Bevezetés

A digitális oktatás fejlődése az elmúlt pár éveben egyre nagyobb hangsúlyt kapott, különösen az interaktív és vizuális tanulási módszerek alkalmazása terén. A hagyományos oktatási módszerek mellett egyre fontosabbá váltak azok a szoftveres megoldások, amelyek játékos módon támogatják a tanulók ismeretszerzését. Igaz ez a természettudományos tantárgyak esetében, ahol az elméleti tudást a vizuális megvalósítás is támogatja

A kémia oktatásában a kísérletek és reakciók vizuális megfigyelése és kipróbálása kifejezetten fontos, hiszen ezek segítségével a tanulók könnyebben megérthetik az egyes anyagok tulajdonságait. Ugyanakkor az iskolai laboratóriumi környezet sok esetben korlátozott lehet. Akár az eszközök hiánya, a veszélyes anyagok használata, vagy az idő- és költségigény gyakran megnehezíti a gyakorlati oktatást.

Általános iskolában illetve gimnáziumban a saját bőrömön is megismerhettem a labori környezetet amelyet főként csak a specifikus irányba továbbtanuló diákok használhattak többségben. Azonban ha kísérletezésre került sor az óra keretein belül, az jelentős felkészülést és időt vett el a tanórából. Ekkor értettem meg igazán miért is az elméleti oktatásra fókuszálnak a pedagógusok.

Ezért is szakdolgozatom célja egy olyan oktatási célú játékalkalmazás létrehozása lett, amely virtuális laboratóriumi környezetben teszi lehetővé különböző kémiai reakciók szimulációját időtakarékosan. Az alkalmazás elsődleges célja, hogy támogassa a tanulók kémiai ismereteinek elmélyítését, valamint játékos formában segítse az összefüggések megértését. E mellett fontosnak tartottam azt is, a diákok akár maguk is próbálkozhassanak tanári segítség nélkül az órán tanult vegyületek készítésének tanulásával.

Az programom lehetőséget biztosít különböző anyagok és eszközök kiválasztására, azok játékmezőn történő elhelyezésére, valamint az elemek közötti reakciók automatikus felismerésére. A vizuális megjelenítés és az interaktív használat elősegíti a tanulók memóriájának fejlesztését.

Egyik fontos funkció a keresőrendszer, amely támogatja a felhasználót egy adott

anyag vagy reakció létrehozásának folyamatában. Ez különösen hasznos lehet a tanulási folyamat során, mivel lépésről lépésre prezentálja a helyes megoldás sorrendjét.

A dolgozat során bemutatom a fejlesztéshez használt technológiákat, a rendszertervet, az adatbázis felépítését, valamint az alkalmazás megvalósításának folyamatát. Emellett részletesen ismertetem a program működését, a főbb modulokat és azok kapcsolatát.

A témaválasztásomat elsősorban az oktatás és az informatika összekapcsolásának lehetősége motiválta. Lelkes támogatóm volt édesanyám is az ötletben, akinek köszönhetem a szakmai tudását a kémiai reakciók iktatása során.

Véleményem szerint a modern oktatásban egyre nagyobb szerepet kapnak a digitális eszközök, különösen azok az interaktív alkalmazások, amelyek képesek a tanulási folyamatot szemléletesebbé és élményszerűbbé tenni.

A kémia tantárgy sajátossága, hogy az elméleti ismeretek mellett fontos szerepet kap a gyakorlati megvalósítás, mivel a tudat alatti befogadóképessége számos embernél jobb, hogyha nem csak papíron lát egy adott reakciót, hanem saját kezűleg is képes megtapasztalni. A laboratóriumi kísérletek segítik a tanulókat abban, hogy jobban megértsék az anyagok tulajdonságait, valamint a különböző reakciók sorrendjét. Ugyanakkor a valós laboratóriumi környezet használata sok esetben korlátozott, mivel speciális eszközöket, anyagokat és biztonságos körülményeket igényel.

Másrészt azért tartottam különösen fontosnak, mert egy virtuális laboratóriumi környezet lehetőséget biztosít arra, hogy a tanulók veszélytelen, könnyen hozzáférhető és költséghatékony módon gyakorolhassák a kémiai reakciókat. Ezen kívül a játékos megközelítés motiváló hatással lehet a diákokra, különösen a fiatalabb korosztály számára.

2. fejezet

Technológiai áttekintés

2.1. Programozási nyelvek

2.1.1. Python

A Python[2] egy magas szintű objektumorientált programozási nyelv, amelyet Guido van Rossum fejlesztett ki az 1980-as évek végén. A fejlesztést az ABC programozási nyelv utódjaként kezdte meg. Első megjelenése 1991-ben történt. Előtérbe helyezi a kód olvashatóságát, egyszerűségét és a vele való munka megkönnyítését. Szintaxisa letisztult, könnyen érthető. A Python név Monty Python-ról született meg, ugyanis Guido van Rossum egyik kedvenc humoristája volt.

Manapság főként mesterséges intelligenciához, gépi tanuláshoz, játékfejlesztéshez használják

Egyetemi éveim alatt több programozási nyelvel is megismerkedtem, ilyenek például a C++[12], C-Sharp[13] vagy Java[14], azonban fő programozási nyelvként a Python-t választottam, amellyel pályafutásomat kezdtem már gimnáziumban.

Python előnyei:

- **Egyszerű szintaxis:** A Python kódja könnyen olvasható és érthető, ami megkönnyíti a vele való munkát.
- **Gyors fejlesztés:** A dinamikus típusrendszer lehetővé teszi, hogy a változók típusait nem szükséges előre deklarálni.
- **Tágas könyvtárkészlet:** A Python közösségben számos könyvtár elérhető a különféle szükségekhez.
- **Keresztplatformosság:** A Python programok Windows, Linux és macOS rendszereken azonos módon futtathatóak.
- **Oktatási célokra alkalmas:** A Python széles körben használt oktatási nyelvként, amely ezáltal ideális egy hasonló célú alkalmazáshoz.

Korlátai

- **Teljesítmény:** A Python lassabb, mint a fordított nyelvek (C++, Java), de az oktatási alkalmazásokhoz elegendő.
- **Memória-felhasználás:** a runtime nagyobb memóriát igényel, mint az alacsonyabb szintű nyelvek.
- **Telepítési követelmények:** Szükséges a Python futtatókörnyezet telepítése.

2.1.2. SQL

A Structured Query Language (röviden SQL[15]) az 1970-es években, IBM által elkezdett adatbázis kezelésre fejlesztett nyelv. Főbb lehetőségei a lekérdezés, módosítás, törlés illetve adatbázis létrehozása és kezelése.

Szakdolgozatom során a már egyetemen megismert MySQL-el dolgoztam, amely lehetővé tette a gyors és egyszerű adatbázis létrehozást.

2.2. GitHub

A GitHub[7] egy webalapú platform, szolgáltatás mely a Git[9] verziókezelő rendszerre épül. Fontos szerepet játszik a kód online tárolásában és elérhetőségében. Lehetőséget ad több fejlesztő együttműködésére és a kód verziókezelésére.

A fejlesztést lokálisan végeztem, ezáltal a Git számomra az a kód elérhetősége és letölthetősége során vált fontossá. **A Git lehetővé teszi:**

- A kód változásainak követését (commit history).
- Az előző verziókhoz való visszatérést (checkout).
- Helyi backup-okat és az adatvédelemben if fontos szerep lát el.

A projekt elérhetővé tételéhez a GitHub platformot alkalmaztam, amellyel lehetőve vált:

- **Nyílt forráskód:** Az alkalmazás forráskódja publikusan elérhető a GitHub repository-ban, amely megkönnyíti az oktatási célú felhasználást.
- **Dokumentáció:** Minden fontos információ, például telepítési útmutató, megtalálható.
- **Hozzáférhetőség:** A tanárok és diákok bármikor letölthetik és futtathatják az alkalmazást saját gépükön.
- **Közösségi visszajelzés:** Az Issues funkcióval felhasználók jelezhetik a hibákat vagy javaslatokat tehetnek a program bővítésére.

2.3. Fejlesztői környezetek

2.3.1. PyCharm

A projekt megvalósítása során elsődleges fejlesztői környezetként a **PyCharm**[4] integrált fejlesztői környezetet (IDE) alkalmaztam, amelyet a JetBrains fejlesztett kifejezetten Python alapú projektekhez. A választás egyik fő oka ismét gimnáziumi éveimbe nyúlik vissza, ahol megismertem a fejlesztőkörnyezetet és megszerettem.

A PyCharm számos olyan funkciót biztosít, amely jelentősen megkönnyítette a fejlesztési folyamatot. Ezek közé tartozik a szintaxiskiemelés, a hibák valós idejű jelzése, valamint a refaktorálási lehetőségek. Ezek az eszközök hozzájárultak a kód olvashatóságának javításához és a hibák számának csökkentéséhez.

A grafikus alkalmazás fejlesztése szempontjából külön előnyt jelentett, hogy a PyCharm jól támogatta a PyGame[3] könyvtár használatát, valamint megkönnyítette a moduláris projektstruktúra kezelését.

2.3.2. Visual Studio Code

A Visual Studio Code (röviden VS Code) egy a Microsoft által létrehozott ingyenes és nyílt forráskódú kód-szerkesztő. Első verziója 2015-ben jelent még, azóta egyik legismertebb fejlesztői eszközzé vált.

Számomra legfőbb előnye széles körű bővítményválasztéka. Fontos tulajdonsága az is, hogy több, különböző fájlt is képes kezelni, ilyenek például a .tex, .py, .txt.

Integrálja a Git verziókezelést, azáltal támogatva a projektek fejlesztését.

Ahogyan a Python, a VS Code is támogatja a crossplatformot, használható Windows, Linux és macOs rendszereken is.

2.3.3. PhPMyAdmin

A phpMyAdmin[6] egy ingyenes, webalapú adatbázis-kezelő eszköz, amelyet PHP nyelven fejlesztettek ki. Elsősorban a MySQL adatbázisok grafikus kezelésére szolgál.

Grafikus megjelenése miatt elősegíti az adatbázisok vizuális megjelenését és azok módosításait. Nem szükséges SQL alaptudás hozzá, manuálisan is kezelhető.

Böngészőben futtatható felület, amely a háttérben egy MySQL szerverként dolgozik, XAMPP[5] rendszerekben elterjedt.

2.3.4. Elérhetőségi linkek

A projekt elérhetőségét az alábbi helyen biztosítom:

GitHub repository: `https://github.com/MRekaa/EY52H0_Szakdolgozat`

2.4. Használt csomagok

2.4.1. Pygame könyvtár

A grafikus megjelenítés és a felhasználói interakciók megvalósításához a Pygame[18] könyvtárat használtam.A Pygame egy Python alapú, nyílt forráskódú fejlesztői könyvtár, amelyet főként 2D-s játékok és grafikus alkalmazások készítésére használnak.

A könyvtár a *Simple DirectMedia Layer (SDL)*[19] multimédiás keretrendszerre épül, amely stabil alapot biztosít a grafikai megjelenítéshez, az eseménykezeléshez, valamint a hang- és képfeldolgozáshoz. Ennek köszönhetően a Pygame széles körben alkalmazható interaktív alkalmazások fejlesztésére.

A szakdolgozat fejlesztése során a Pygame megfelelő választásnak bizonyult, mivel lehetővé tette a 2D-s játékmező egyszerű és hatékony megvalósítását, az objektumok megjelenítését, a sprite-ok megjelenítését valamint a drag-and-drop jellegű felhasználói interakciók kezelését.

PyGame tulajdonságai

- **Keresztplatformosság:** A Pygame támogatja a Windows, Linux és macOS operációs rendszereket, így az alkalmazás különböző környezetekben is futtatható.
- **Széles körű támogatás:** Nagy fejlesztői közösséggel rendelkezik, ezért számos dokumentáció, útmutató és oktatóanyag érhető el hozzá. ez.
- **Játékfejlesztési funkciók:** A Pygame támogatja a sprite-kezelést, ütközésvizsgálatot és eseménykezelést, amelyek elengedhetetlenek voltak a projekt megvalósításához.

2.4.2. MySQL-connector-python

Az adatbázis-kapcsolat kezeléséhez a mysql-connector-python[11] csomagot alkalmaztam. Ez egy Python nyelvhez készült hivatalos MySQL klienskönyvtár, amely lehetővé teszi a Python alkalmazás és a MySQL adatbázis közötti kommunikációt.

A könyvtár segítségével a program képes kapcsolatot létesíteni az adatbázissal, SQL lekérdezéseket végrehajtani, valamint az ott tárolt adatokat feldolgozni. A projekt során ezt a csomagot használtam az anyagok, eszközök és kémiai reakciók adatainak lekérdezésére és betöltésére.

A `mysql-connector-python` könyvtár előnye, hogy stabil és megbízható adatbázishozzáférést biztosít, valamint támogatja a hibakezelést is, például sikertelen kapcsolódás vagy hibás SQL lekérdezés esetén.

A csomag használata lehetővé tette, hogy az alkalmazás adatvezérelt módon működjön, így az új anyagok és reakciók hozzáadása egyszerűen, az adatbázis bővítésével megvalósítható, a programkód módosítása nélkül.

A mysql-connector-python a projektre legmegfelelőbb választás, mivel közvetlenül és megbízhatóan csatlakozik a MySQL adatbázishoz.

3. fejezet

Rendszerterv

3.1. A rendszer célja

A Python alapú 2D-s rendszer a kémiai reakciók játékos tanulását támogatja, lehetővé téve a diákok számára a vizuális gyakorlást egy számítógépes labor környezetben.

A játék lehetőséget biztosít különböző anyagok és eszközök kombinálására, amelyek segítségével új vegyületek hozhatók létre. Ez a folyamat hozzájárul a tanulók memóriájának fejlesztéséhez, valamint az összefüggések könnyebb megértéséhez. Az anyagok vizuális megjelenítése továbbá elősegíti a tudat alatti, könnyebb és hatékonyabb tanulási folyamatot.

3.2. Követelmények listája

3.2.1. Funkcionális követelmények

- Felhasználó kiválaszthat anyagokat és eszközöket a hotbar-ból.
- A hotbarban az eszközök és elemek külön választva helyezkednek el, amelyek között válthat a felhasználó.
- Itemeket helyezhet a játékmezőre drag-and-drop módon.
- Automatikus reakciók indulnak, ha két item megfelelő távolságon belül van.
- Ha két item nem képes reakcióra nem történik változás.
- Keresőrendszer biztosítja a lehetséges reakciók megtalálását.
- Itemek törölhetők a trash zónába helyezéssel.
- Reszponzív felület, amely alkalmazkodik az ablakméret változásához.

3.2.2. Nem funkcionális követelmények

- Adatbázis alapú reakciók tárolása MySQL-ben.
- Egyszerű, intuitív felhasználói felület Pygame-el megvalósítva.
- Hatékony teljesítmény kis erőforrásigénnyel.
- Könnyen bővíthető adatbázis.

3.3. Használati esetek (Use case-ek)

- **Item spawnozása:** A felhasználó kiválaszt egy anyagot vagy eszközt a hotbarból, és a játékmezőre helyezi.
- **Reakció kiváltása:** Két item közel helyezésekor automatikusan létrejön az új anyag, ha van megfelelő reakció.
- **Recept keresése:** A felhasználó beírja a keresett anyag nevét, és megjelennek a lehetséges kombinációk.
- **Item törlése:** Az itemet a trash zónába húzva eltávolítja a játékból.

3.4. Adatbázis-terv

Az adatbázis MySQL alapú, és legalább harmadik normálformában (3NF) van tervezve. A fő táblák:

- Tárolja a reakciókat (item1, item2, result, equation, type). Az item1 és item2 kombinációja hozza létre a result, eredmény anyagot. Az equation tárolja a kémiai reakció egyenletét, a type adja meg a result típusát.
- Az elérhető anyagok listája (name).
- Az elérhető eszközök listája (name).

3.5. UML-diagramok, folyamatábrák

3.5.1. Osztálydiagram

Fő osztályok:

- Main (fő vezérlő)
- ItemOnBoard (itemek a játékmezőn)

- Reaction (reakciók kezelése)
- Hotbar (elérhető anyagok, tárgyak)
- SearchSystem (keresési lehetőség)
- Trash (itemek törlése)

Az ItemOnBoard kapcsolatban áll a Reaction-nal, a Hotbar-ok öröklik a Hotbar osztályt.

3.5.2. Folyamatábra

A játék fő ciklusa: Eseménykezelés (egérkattintás, mozgatás), itemek frissítése, reakciók ellenőrzése, renderelés.

4. fejezet

Saját szoftver megvalósítása

4.1. Az alkalmazás felépítése

4.1.1. Architekturális áttekintés

Az alkalmazás moduláris felépítésű, MVC-szerű (Model-View-Controller) architektúrával épül fel. Ez a megközelítés elválasztja az adatkezelést (Model), a felhasználói felületet (View), valamint az üzleti logikát (Controller), ezáltal biztosítva a kód szervezettségét, módosíthatóságát és karbantarthatóságát.

A rendszer három fő rétegből áll:

1. **Adatkezelési réteg (Data Layer):** Az adatbázissal való kommunikációt és az adatok betöltését kezeli. Ez a réteg a MySQL adatbázissal kapcsolódik a `mysql-connector-python` könyvtáron keresztül.

2. **Üzleti logikai réteg (Business Logic Layer):** Az alkalmazás magját képezi, ide tartozik a játéklogika, a kémiai reakciók ellenőrzése, az objektumok közötti interakciók kezelése.

3. **Prezentációs réteg (Presentation Layer):** A Pygame keretrendszer felhasználásával valósul meg, kezeli a grafikus megjelenítést, a felhasználói interakciókat és a vizuális visszajelzéseket.

4.1.2. Komponensek és azok szerepe

Az alkalmazás fő komponensei az alábbiak:

Main osztály (main.py)

Az alkalmazás belépési pontja és fő vezérlője. Ez a komponens koordinálja a játékciklust (game loop). A Main osztály felelős az alábbiak kezeléséért:

- A Pygame inicializálása és az ablak létrehozása
- Az összes többi komponens inicializálása (Hotbar, Reaction, SearchSystem, Trash)
- Az eseménykezelés: egérkattintások, egérmozgás, ablak átméretezés, billentyűlenyomások
- A játékciklus logikájának menedzselése (input feldolgozás, update, render)
- Az alkalmazás leállításának kezelése

A Main osztály szinkronizálja az összes többi modul működését, biztosítva, hogy minden komponens szinkronban legyen a játékciklus ritmusával.

ItemOnBoard osztály (itemOnBoard.py)

Az a játékmezőn jelenlévő objektumokat (anyagok és eszközök) reprezentálja. Ez az osztály a Model komponensként funkcionál az MVC architektúrában. Fő feladata:

- Az egyes itemek adatainak tárolása: pozíció, típus (anyag/eszköz), méret, kép
- Az items pozíciójának frissítése drag-and-drop műveletek során
- Az items megjelenítésének koordinálása a Pygame-gal
- Ütközésdetekció: annak meghatározása, hogy mely itemek kombinálódnak[17]
- Az objektumok közötti lehetséges reakciópárok azonosítása

Az ItemOnBoard osztály közvetlenül kommunikál a Reaction modullal az ütközésdetekció eredményeit átadva, és a View réteg felé a Pygame-on keresztül a megjelenítéshez szükséges adatokat nyújtja.

Reaction osztály (reaction.py)

Az üzleti logika egyik központi komponense, amely a kémiai reakciók magját képezi. Főbb felelősségei:

- Az ItemOnBoard-tól érkező távolságadatok alapján annak ellenőrzése, hogy mely objektumok között lehetséges reakció
- Az adatbázisban tárolt reakcióadatok lekérdezése a `config.py`-n keresztül
- A megfelelő feltételek teljesülése esetén az új anyagok létrehozása
- A reakció során az eredeti objektumok eltávolítása és az új objektum hozzáadása

A Reaction osztály az ItemOnBoard-dal szoros kapcsolatban áll, mivel az ItemOnBoard az új anyagokat adja hozzá a játékhoz a reakció után. Ez a komponens az üzleti logika magját reprezentálja, amely a kémiai ismereteket implementálja.

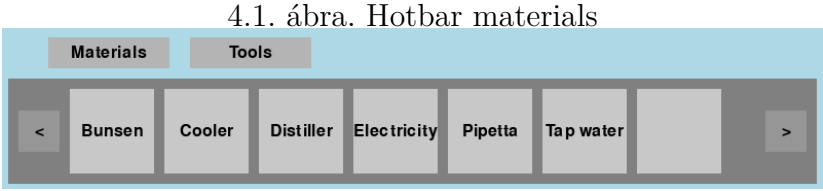
Hotbar komponensek (HB/ mappa)

A felhasználói felület eszköztárát valósítja meg, ahol a felhasználók kiválaszthatják az elérhető anyagokat és eszközöket. A két lehetőséget a 4.1. és 4.2. ábra mutatja be. A modulok struktúrája:

- **hotbar.py**: Az alap hotbar funkciót biztosítja, kezeli az eszköztár pozícióját, méretét és a reszponzivitást az ablakméret-változásokra.
- **materialsH.py**: Az elérhető anyagok listáját jeleníti meg egy léptethető sávban, az adatbázisból betöltött adatok alapján.
- **hotbar.py**: Az elérhető eszközöket jeleníti meg hasonló módon.

Ezek a komponensek kommunikálnak a `config.py`-vel az adatok lekérdezéséhez, és az ItemOnBoard-dal az új objektumok létrehozásához, amikor a felhasználó kiválaszt egy anyagot vagy eszközt.

4.1. ábra. Hotbar materials

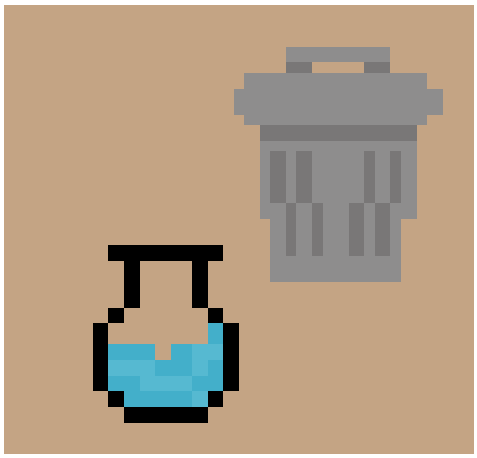


4.2. ábra. Hotbar tools

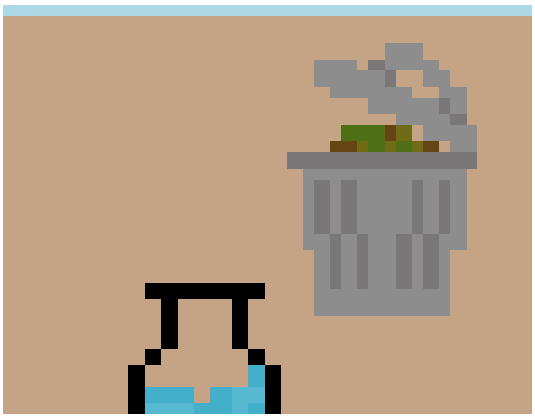
Trash komponens (trash.py)

Az objektumok törlésének funkcionáltatásáért felelős trash osztály. Alapvető feladatai közé tartozik a törlési zóna megjelenítése és pozícionálása az ablak egy rögzített helyén (jobb felső sarkokban). Az egérmozgás figyelése és annak detektálása, hogy mely objektumok vannak a törlési zónában Visual feedbacket ad, a kuka sprite-jának frissítésével ha egy adott item a zónába kerül. A változást a 4.3. illetve 4.4. ábra szemlélteti.

Ez a komponens szoros kapcsolatban áll az ItemOnBoard-dal, mivel a kuka sprite zónájába kerülő objektumok eltávolításra kerülnek az ItemOnBoard listájából, ezáltal törölve lesznek a játékfelületről is.



4.3. ábra. Kuka lezárva



4.4. ábra. Kuka nyitva aktív item-mel a közelében

SearchSystem modul (searchSystem.py)

A keresőrendszert implementálja, amely segíti a célzott anyagok előállításában. Funkciói:

- A felhasználó által megadott anyag nevéből indulva az adatbázisban történő keresés
- A lehetséges receptek megjelenítése, amelyek az adott anyaghoz vezetnek

Config modul (config.py)

Fontos szerepet kap az adatbázis összeköttetés során:

- Az alkalmazás indításakor betölti az összes adatbázis-adatot (anyagok, eszközök, reakciók)
- Egy globális adatstruktúrát hoz létre az alkalmazás memóriájában, amely gyors hozzáférést biztosít
- Az adatbázis-hibák kezelése (MySQL szerverhez való kapcsolódás sikertelen)
- Az adatok frissítésének és szinkronizálásának kezelése

Tool/materials osztály (tool.py/materials.py)

Az objektumokat definiáló osztályok. A 4.1. ábra a tools osztályt mutatja be.

```
class ToolsH(Hotbar):
    def __init__(self, window_size, game):
        super().__init__(window_size, game)
        self.items = AVAILABLE_TOOLS
        self.type = ITEM_TYPE_TOOL
```

```
        self.active_tab = "Tools"
```

Listing 4.1. Tool osztály

Settings modul (settings.py)

Az alkalmazás beállításait és konstansait tárja:

- Grafikai beállítások: színek, betűtípusok, margók
- Az ablak alapmérete és reszponzivitási szabályai
- Konstans értékek: fizikai paraméterek
- A reszponzívitás kiszámításának formulái

Ez a modul az alkalmazás egészében használt globális konfigurációs adatokat tartalmaz.

4.1.3. Adatfolyam és működési logika

Az alkalmazás működése az alábbi sorrend szerint történik:

1. Az alkalmazás indításakor a `config.py` modul betölti az adatbázisból az összes rendelkezésre álló anyagot, eszközt és kémiai reakciót.
2. A `main.py` inicializálja a Pygame-ot és elindítja a fő játékciklust.
3. Minden frame-ben a rendszer feldolgozza az felhasználói eseményeket: egér eseményeket, mozgásokat és drag-and-drop operációkat.
4. Az ItemOnBoard komponens frissíti az objektumok pozícióit a felhasználói inputok alapján.
5. A Reaction modul folyamatosan ellenőrzi az objektumok közötti távolságot, és ha az előfeltételek teljesülnek, reakciókat indít el.
6. A Pygame végül rendereli a frissített állapotot a képernyőre.
7. Az Trash és SearchSystem modulok szükség szerint végzik el kiegészítő funkcióikat.

Pygame keretrendszer integrációja

A Pygame egy Python alapú grafikus könyvtár, amely SDL (Simple DirectMedia Layer) fölött működik. Az alkalmazásban a Pygame a következő feladatokat végzi:

- **Ablak és megjelenítés:** A Pygame kezeli az alkalmazás ablakát, a felbontást és az átméretezést. Az `pygame.display` modul kezelő feladata az ablakhoz való rajzolás és az image renderelés.
- **Eseménykezelés:** A Pygame az `pygame.event` modulon keresztül kezeli az összes felhasználói inputot:
  - Egérkattintások (`MOUSEBUTTONDOWN`, `MOUSEBUTTONUP`)
  - Egérmozgás (`MOUSEMOTION`)
  - Ablakátméretezés (`VIDEORESIZE`)
  - Billentyűlenyomások (`KEYDOWN`, `KEYUP`)
- **Grafikus elemek és spritok:** A Pygame a `pygame.sprite` osztályokkal kezeli az objektumokat. Az alkalmazásban az ItemOnBoard osztályok Pygame spriteokból örökölnek, amely megkönnyíti az kombinációkat és a megjelenítést.

Adatbázis-kezelés és MySQL integrációja

Az alkalmazás MySQL adatbázist használ. Az integráció:

- **mysql-connector-python csomag:** Ez a csomag a MySQL szerver és a Python alkalmazás közötti kommunikációt kezelik. Az `dataBase.py` modul az alábbi műveleteket végzi:
  - Kapcsolódás a MySQL szerverhez felhasználónév és jelszó alapján
  - SQL lekérdezések végrehajtása: SELECT, INSERT, UPDATE, DELETE
  - Hibakezelés: kapcsolódási hibák, lekérdezési hibák, adatkonverziós hibák
- **Adatbetöltési stratégia:** Az alkalmazás indításakor a `config.py` modul az összes adatbázis-adatot betölti az alkalmazás memóriájába. Ez a megközelítés:
  - Előnye: Gyors adathozzáférés futás közben, mivel az adatok a memóriában vannak
  - Hátránya: Az alkalmazás memória-igénye növekszik, és az adatbázis módosítása szükséges az alkalmazás újraindításához

Komponensek közötti kommunikáció

Az alkalmazás komponensei így kommunikálnak:

- **Main → Config:** Az alkalmazás indításakor a Main osztály meghívja a Config modult az adatok betöltéséhez.
- **Main → ItemOnBoard:** A Main osztály kezeli az egéresemények feldolgozását és továbbítja azokat az ItemOnBoard osztálynak.
- **ItemOnBoard → Reaction:** Az ItemOnBoard objektumokat hoznak létre és nyomon követik, majd az kombinációk után átadja az információkat a Reaction modulnak.
- **Reaction → Config:** A Reaction modul lekérdezi a lehetséges recepteket a Config modulból.
- **Hotbar → ItemOnBoard:** A felhasználó által kiválasztott anyagok alapján az ItemOnBoard új objektumokat hoz létre.
- **Trash → ItemOnBoard:** Az objektumok törlésének műveletét az ItemOnBoard listájából hajtja végre.

Hibatűrés és hibakezelés

Az alkalmazás hibalehetőségeit biztosítják:

- **Adatbázis-hiba kezelés:** Ha a MySQL szerver nem érhető el az indítás során, a `dataBase.py` hibaüzenetet jelenít meg, és az alkalmazás üres listákkal próbál továbblépni.
- **Képbetöltési hibák:** Ha egy kép nem található, a Pygame egy alapértelmezett nem a képet jeleníti meg, hanem az item nevét.
- **Ablak-átméretezési hibaelkerülés:** A Settings modul biztosítja, hogy az ablak átméretezésekor az összes komponens a helyes pozícióban maradjon.

Bővíthetőség és fenntarthatóság

Az alkalmazás moduláris szerkezete lehetővé teszi az egyszerű bővítést:

- **Új anyagok,reakciók hozzáadása:** Az adatbázisban egy új sor felvétele bármely táblában elegendő az új itemek, reakciók integrálásához.
- **Új funkciók:** Az új modulok egyszerűen hozzáadhatók a meglévő szerkezethez.

4.2. Adatbázis

Kapcsolatteremtés az adatbázissal

A backend az alkalmazás szerveroldali logikájáért és az adatok kezeléséért felel. A rendszer az adatbázis-műveletekhez MySQL adatbázist használ, amelyhez a kapcsolat a `mysql-connector-python` könyvtár segítségével valósul meg.

Az adatkezelési réteg két fő modulra oszlik. A `config.py` modul felel az alkalmazás indulásakor szükséges alapadatok betöltéséért, ilyenek a reakciók és az itemek adatbázisból történő lekérdezését. Ez biztosítja, hogy a játék logikája dinamikusan, az adatbázis tartalmára alapozva működjön.

A `database.py` (dataBase.py) modul látja el az adatbázis-hozzáférési réteg szerepét, amely tartalmazza az SQL lekérdezések végrehajtását, valamint az adatok kiolvasását. Ez a réteg elválasztja az adatbázis-kezelési logikát az alkalmazás többi részétől, ezáltal növelve a rendszer modularitását és karbantarthatóságát.

Az adatbázis felépítése lehetővé teszi a játékban használt adatok könnyű bővíthetőségét, mivel az új itemek és reakciók közvetlenül az adatbázis módosításával integrálhatók, a programkód megváltoztatása nélkül.

A backend felelős a játéklogika futtatásáért, beleértve a reakciók ellenőrzését és az objektumok közötti interakciók kezelését. A rendszer folyamatosan figyeli a játékterületen lévő elemek közötti távolságot, és meghatározott feltételek teljesülése esetén reakciókat indít el.

A háttérfolyamatok eseményvezérelt módon működnek, amelyet a PyGame fő ciklusa (game loop) hajt végre. Ez biztosítja, hogy a játék állapota minden frame-ben frissüljön, miközben az adatkezelés és a logikai műveletek elkülönítve, mégis szinkronban történnek.

Adatbázis-struktúra:

Az alkalmazás három táblát használ, amelyeket a 4.5. ábra prezentál egy dbdiagrammal[10]:

- **reactions:** A kémiai reakciókat definiálja (item1,item2,result,equation,type). Item1 és item2 mező a reakcióhoz használt bemeneti elemeket tárolja, amelyekkel létrejön egy adott típusú eredmény(result). A folyamat képletét az equation tárolja el.
- **tools:** Az eszközök nevét tartalmazza.
- **materials tábla:** A kémiai anyagok nevét tárolja.

Az adatbázis kapcsolatokat nem tartalmaz, mivel a további feldolgozást a program végzi.

4.5. ábra. Adatbázis felépítése

4.3. Frontend

A frontend megvalósítása a PyGame könyvtár felhasználásával történt, amely egy Python alapú, 2D grafikus alkalmazások fejlesztésére alkalmas keretrendszer. A rendszer főbb feladatai közé tartozik a felhasználói interakciók kezelése, a grafikus elemek megjelenítése, valamint a játéklogika vizuális visszacsatolásának biztosítása.

Az alkalmazás támogatja a drag-and-drop mechanizmust, amely lehetővé teszi az objektumok interaktív mozgatását a játéktéren belül. Emellett a felület reszponzív kialakítással rendelkezik, amely biztosítja, hogy az ablakméret változásakor a grafikus komponensek elrendezése és méretezése dinamikusan alkalmazkodjon az aktuális képernyőméretthez.

A reszponzív működés alapját egy központi layout-számítási mechanizmus képezi, amely az aktuális ablakméretből (szélesség és magasság) származtatja a felhasználói felület fő elemeinek, például a játéktérnek és a hotbar komponenseknek méreteit. Az átméretezési események kezelése során a rendszer újraszámolja a vizuális elemek pozícióit, így biztosítva az arányos megjelenítést különböző felbontások esetén is.

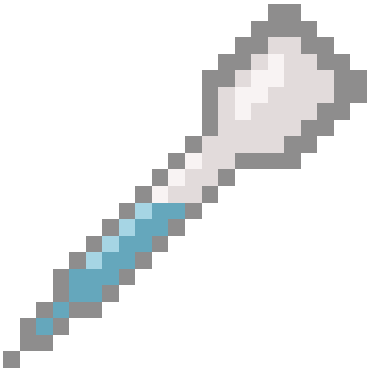
A vizuális visszajelzések között tooltip-ek, valamint reakcióláncok (history chain) megjelenítése szerepel, amelyek segítik a felhasználót a játékbeli interakciók követésében és értelmezésében.

Képek

A játékban használt grafikai elemeket a Piskel[1] online szerkesztő segítségével készültek. Az sprite-ok PNG formátumú, átlátszó hátterű képek, amelyek könnyedén felhasználhatóak a PyGame segítségével.

Ilyen sprite a 4.6. ábrán látható pipetta eszköz is amely víz felvétele után a 4.7. ábra látható már anyagként.

4.6. ábra. Pipetta sprite



4.7. ábra. Pipetta(H2O) sprite

5. fejezet

Futtatás

A program futtatásához a következő lépések szükségesek:

- Python 3.10 telepítése és virtuális környezet létrehozása a projekt mappájában.
- A szükséges csomagok telepítése a virtuális környezetbe: `pygame` és `mysql-connector-python`.
- MySQL szerver futtatása a helyi gépen, és a `labor` nevű adatbázis létrehozása.
- Az adatbázishoz tartozó táblák: `reactions`, `materials`, .
- A `src/config.py` modul beolvassa az adatokat az adatbázisból, így a táblák helyes struktúrája és tartalma szükséges a program működéséhez.
- Ha a `mysql-connector-python` csomag hiányzik, vagy a MySQL nem elérhető, a `src/dataBase.py` hibaüzenetet ad, és a betöltés üres lista lesz.

Az alkalmazás indítása előtt aktiválni kell a virtuális környezetet, majd a fő scriptet kell futtatni:

- PowerShell-ben a környezet aktiválása: `.\venv\Scripts\Activate.ps1`
- A játék elindítása: `.\venv\Scripts\python.exe src/main.py`
- Ha a virtuális környezet már aktív, elég közvetlenül futtatni: `python src/main.py`

Ha nincs `requirements.txt`, akkor a csomagokat kézzel kell telepíteni a fent említett módon.

6. fejezet

Tesztelés

6.1. Manuális tesztelés

Funkció	Leírás	Opciók/Példák
Hotbar váltás	Sikeres	Gombok megfelelően váltanak hotbar-t
Material helyezése a játékfelületre	Sikeres	A material megjelent a a játékfelületen
Tool helyezése a játékfelületre	Sikeres	A tool megjelent a a játékfelületen
Adatbázis betöltése	Sikeres	Sikeresen betöltöttek az adatok
Két material reakciója	Sikeres	Új material jött létre, sikeres
Material és tool reakciója	Sikeres	Sikeres reakció
Kuka megjelenése	Sikeres	Kuka jó helyen jelent meg
Kuka reszponzívan tartja a helyét	Sikeres	Kuka jó helyen jelent meg
Kuka sprite váltása ha adott item van a közelében	Sikeres	A kuka megfelelően vált sprite-ot
Keresés jó eredményeket ad ki	Sikertelen (javítva)	A keresés nem ad ki nemlétező eredményeket
Az itemek pozícionálása ablakméret váltáskor	Sikeres	Az itemek megfelelő helyen vannak ablakméret váltáskor
A hotbár reszponzív	Sikertelen (javítva)	Nagyításkor eltűnik - rossz pozícionálás, javítva
Itemek adatainak kiíratása	Sikeres	Megfelel adatok jelennek meg az itemek alatt
Hotbar léptetése a materials itemeknél	Sikeres	Összes item megtekinthető
Sprite-ok megjelnése	Sikeres	A sprite-ok megfelelően jelennek meg

6.1. táblázat. Manuális teszt jegyzőkönyv

6.2. Automatikus tesztelés

Az automatikus tesztek megvalósításához a Mockito [16] keretrendszert alkalmaztam. A Mockito egy népszerű Java-alapú mocking könyvtár, amely lehetővé teszi tesztkörnyezetben helyettesítő objektumok létrehozását tényleges függőségek helyett.

Funkció	Leírás	Opciók/Példák
get_recipe() függvény működése H2O-ra, NaCl-ra CO-ra,CO2-ra,	Sikeres	Megfelelően visszadja a kiinduló alapanyagokat
can_react() függvény működése H2+O2-re	Sikeres	Sikeres reakciótalálat
can_react() függvény negatív teszteset Na+Cl-re	Sikeres	Sikeres, nem talált reakciót
check() függvény tesztelése	Sikeres	Az anyag létrejött
check() függvény tesztelése több kimenetre	Sikeres	Reakció után megfelelő számú anyag spawnolt
reaction() függvény tesztelése, használt itemek törlése	Sikeres	Reakció után az itemek törlődnek
delete_item() függvény tesztelése	Sikeres	Item sikeresen törlődik
trash_hover() függvény működése	Sikeres	Megfelelően működik
trash_draw() függvény működése	Sikeres	Megfelelően kirajzolódik

6.2. táblázat. Automatikus tesztelési jegyzőkönyv

A Mockito használatával a függőségek viselkedése előre definiálható, valamint ellenőrizhető, hogy egy adott metódus milyen paraméterekkel és hányszor került meghívásra. Ez növeli a tesztek megbízhatóságát és olvashatóságát.

7. fejezet

Összegzés

A szakdolgozat célja egy Python alapú, grafikus felülettel rendelkező oktatási alkalmazás megvalósítása volt, amely a kémiai reakciók tanulását játékos és vizuális formában támogatja. A fejlesztés során egy olyan rendszer létrehozása került előtérbe, amely nem csupán bemutatja a reakciókat, hanem interaktív módon lehetőséget biztosít azok kipróbálására és megértésére is.

A megvalósított alkalmazás több szempontból is sikeresen teljesítette az eredetileg kitűzött követelményeket. A Pygame alapú grafikus felület biztosítja a felhasználói interakciókat, míg a MySQL adatbázis háttér lehetővé teszi a reakciók és anyagok rugalmas kezelését. A moduláris, MVC-szerű architektúra hozzájárult a rendszer átláthatóságához és bővíthetőségéhez, amely különösen fontos egy oktatási célú szoftver esetében.

A fejlesztés során kiemelt szerepet kapott a felhasználói élmény kialakítása. A drag-and-drop mechanizmus, a reszponzív felület, valamint a vizuális visszajelzések mind azt a célt szolgálták, hogy a tanulási folyamat minél intuitívabb és motiválóbb legyen. A tesztelési fázis során megfigyelhető volt, hogy a felhasználók gyorsan elsajátítják az alkalmazás használatát, és a vizuális elemek jelentősen segítik az összefüggések megértését.

A projekt elkészítése során számos technikai és tervezési kihívással kellett szembenézni, például az adatbázis és a játéklogika szinkronizálásával, valamint a valós idejű interakciók hatékony kezelésével. Ezek megoldása hozzájárult a programozási és rendszertervezési ismereteim elmélyítéséhez.

A rendszer jelenlegi formájában stabilan működik, azonban több irányban is továbbfejleszthető. Ilyen lehet például a hang- és vizuális effektek bevezetése, a reakciók animálása, valamint egy részletesebb oktatási modul kialakítása, amely lépésről lépésre vezeti végig a tanulókat a kémiai folyamatokon.

Összességében elmondható, hogy a létrehozott alkalmazás nemcsak technikai szempontból sikeres, hanem pedagógiai értéket is képvisel. A projekt rávilágított arra, hogy az informatika és az oktatás hatékony kombinációja jelentős mértékben hozzájárulhat

a tanulási folyamatok modernizálásához és élményszerűbbé tételéhez.

Irodalomjegyzék

[1] Piskel: *Online pixel art és sprite szerkesztő*, https://www.piskelapp.com/ (Letöltés: 2026.04.21.)

[2] Python Software Foundation: *Python 3 hivatalos dokumentáció*, https://docs.python.org/3/ (Letöltés: 2026.04.21.)

[3] Pygame: *Hivatalos Pygame dokumentáció*, https://www.pygame.org/docs/ (Letöltés: 2026.04.21.)

[4] JetBrains: *PyCharm Getting Started – Hivatalos bevezető és útmutató*, https://www.jetbrains.com/help/pycharm/getting-started.html (Letöltés: 2026.04.21.)

[5] Apache Friends: *XAMPP hivatalos letöltési és dokumentációs oldal*, https://www.apachefriends.org/ (Letöltés: 2026.04.21.)

[6] phpMyAdmin: *Hivatalos phpMyAdmin dokumentáció és letöltési oldal*, https://www.phpmyadmin.net/ (Letöltés: 2026.04.21.)

[7] GitHub: *GitHub témák és dokumentációs gyűjtemények*, https://github.com/topics/documentation (Letöltés: 2026.04.21.)

[8] Microsoft: *Visual Studio Code hivatalos dokumentáció*, https://code.visualstudio.com/docs (Letöltés: 2026.04.21.)

[9] Git: *Hivatalos Git dokumentáció*, https://git-scm.com/docs (Letöltés: 2026.04.21.)

[10] dbdiagram.io: *Hivatalos dokumentáció – adatbázis diagramok és DBML leírás*, https://docs.dbdiagram.io/ (Letöltés: 2026.04.21.)

[11] Oracle: *MySQL Connector/Python hivatalos dokumentáció*, https://dev.mysql.com/doc/connector-python/en/ (Letöltés: 2026.04.21.)

[12] DevDocs: *C++ referencia dokumentáció (C/C++ API és standard library leírások gyűjteménye)*, https://devdocs.io/cpp/ (Letöltés: 2026.04.21.)

[13] Microsoft Learn: *C# hivatalos dokumentáció és útmutató (.NET platform)*, https://learn.microsoft.com/en-us/dotnet/csharp/ (Letöltés: 2026.04.21.)

[14] Oracle: *Java Platform, Standard Edition (Java SE) hivatalos dokumentáció*, https://docs.oracle.com/en/java/ (Letöltés: 2026.04.21.)

[15] W3Schools: *SQL Tutorial és referenciaanyag (SQL nyelv alapjai, lekérdezések és példák)*, https://www.w3schools.com/sql/ (Letöltés: 2026.04.21.)

[16] Mockito: *Hivatalos Mockito dokumentáció és felhasználói útmutató*, https://site.mockito.org/ (Letöltés: 2026.04.21.)

[17] ITU Online: *What Is Python Pygame?*, https://www.ituonline.com/tech-definitions/what-is-python-pygame/ (Letöltés: 2026.04.21.)

[18] Dr. M. K. Jayanthi Kannan: *Introduction to Pygame and Game Development using Python Pygame*, ResearchGate, https://www.researchgate.net/publication/377064275_Introduction_to_Pygame_and_Game_ (Letöltés: 2026.04.21.)

[19] Simple DirectMedia Layer (SDL): *Official SDL homepage*, https://www.libsdl.org/ (Letöltés: 2026.04.21.)

Nyilatkozat

Alulírott *Mészáros Réka*, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, *Python alapú grafikus alkalmazásfejlesztés* című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírásommal igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2026. április 21.

Mészáros Réka

aláírás